Informatica® API Center
October 2026

Asset Management

Informatica API Center Asset Management
October 2026

© Copyright Informatica LLC 2022, 2026

This software and documentation are provided only under a separate license agreement containing restrictions on use and disclosure. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC.

U.S. GOVERNMENT RIGHTS Programs, software, databases, and related documentation and technical data delivered to U.S. Government customers are "commercial computer software" or "commercial technical data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation is subject to the restrictions and license terms set forth in the applicable Government contract, and, to the extent applicable by the terms of the Government contract, the additional rights set forth in FAR 52.227-19, Commercial Computer Software License.

A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Portions of this software and/or documentation are subject to copyright held by third parties. Required third party notices and AI assistant acknowledgements are included with the product.

The information in this documentation is subject to change without notice. If you find any problems in this documentation, report them to us at infa_documentation@informatica.com.

Informatica products are warranted according to the terms and conditions of the agreements under which they are provided. INFORMATICA PROVIDES THE INFORMATION IN THIS DOCUMENT "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.

Publication Date: 2026-10-18

Table of Contents

Preface 5

Chapter 1: Project and asset management 6

Projects 6
- Creating a project 6
- Creating a folder 6

Moving folders and assets 7

Renaming projects, folders, and assets 7

Deleting projects, folders, and assets 7

Tags 8
- Creating tags 8
- Assigning tags 9
- Editing and deleting tags 9

Assets 9

Asset dependencies 10

Chapter 2: Asset migration 11

Asset migration requirements 11

Asset migration user roles 12

Dependent assets 12

Asset export 14
- Make assets exportable 14
- Export files 16
- Exporting assets 16
- Exporting duplicate assets 16

Asset import 17
- Importing assets 17
- Asset name conflicts 18
- Rules and guidelines for importing designed APIs 18
- Rules and guidelines for importing published APIs 18
- Rules and guidelines for importing managed APIs 18

Downloading an import or export log 19

Chapter 3: Source control 20

Source control actions 21

Source control and the Git repository 21

Configuring repository access 22

Project-level repositories 22
- Linking a project to a source control repository 22

Source control pulls 23

Project and folder pulls. . . . 23
Pulling an object 24
Commit tags. . . . 24
Checking out and checking in objects. . . . 25
Checking out an object. . . . 25
Checking in an object. . . . 26
Comparing asset versions. . . . 26
Deleting an object. . . . 27
Reverting to an older version. . . . 27
Undoing a checkout. . . . 28
Unlinking an object. . . . 29
Working with multiple objects. . . . 29
Viewing source control columns on the Explore page. . . . 30
Source control best practices. . . . 31

Index. . . . 32

Preface

Read *Asset Management* to learn how to set up, organize, and perform actions on folders and assets.

CHAPTER 1

Project and asset management

You can manage projects and assets on the **Explore** page. The **Explore** page is an IDMC feature that is available for most services. If you use multiple services, you might see projects, folders, and assets for all of your services on the **Explore** page.

You can manage your API Center projects and assets in the following ways:

- Rename projects, folders, or assets.
- Delete projects, folders, or assets.
- Apply tags so you can filter for related assets on the **Explore** page.

Projects

API Center assets are organized in projects. You can create multiple folders within a project to contain the assets that you need for your project.

For example, you might have a business need to provide monthly analyses of your organization's API activity. You create a project called Monthly API Analysis. You create folders that contain the assets that you use on a monthly basis to integrate data from various systems to perform your analysis.

An API Center project can include assets such as APIs and Application Integration processes.

Creating a project

Create a project to store folders and assets on the **Explore** page.

1. On the **Explore** page, click **New Project**.
2. Enter a name for the project, and optionally, enter a description.
3. Click **Save**.

   API Center creates the project. The project appears on the **Explore** page.

Creating a folder

Create a folder to store assets on the **Explore** page.

1. On the **Explore** page, click a project, and then click **New Folder**.
2. Enter a name for the folder, and optionally, enter a description.
3. Click **Save**.

   API Center creates the folder. The folder appears on the project details page.

Moving folders and assets

You can move folders and assets on the **Explore** page.

1. On the **Explore** page, navigate to the folder or assets that you want to move.
2. To move a folder or a single asset, in the row that contains the folder or asset, click **Actions** and select **Move To**.
3. Browse to the new location and click **Select**.

   The asset is moved to the target location.
   If the target location contains an asset of the same name and type, the move fails.

   Note: You can't move a designed API from one location to another.

Renaming projects, folders, and assets

You can rename projects, folders, and assets without losing references to objects that they use.

1. On the **Explore** page, navigate to the asset that you want to rename.
2. To rename an asset, in the row that contains the asset, click **Actions** and select **Rename**.
3. To rename a project or folder, in the row that contains the project or folder, click **Actions** and select **Properties**.
4. Enter the new name and click **Save**.

   The name can contain up to 80 characters, including ASCII letters, Japanese characters, digits, hyphens, and underscores. The name must be unique within the folder.

Deleting projects, folders, and assets

You can delete a project, folder, or asset if you no longer need it. However, before you delete it, verify that no users in the organization plan to use it. You can't retrieve projects, folders, or assets after you delete them. You can't delete a published API.

You can't delete a project, folder, or asset in the following situations:

- The project, folder, or asset is used by another asset. You must first delete the dependencies of the project, folder, or asset before you can delete it.
- The project, folder, or asset has a cyclic dependency on other assets. You must delete all the assets simultaneously. For example, if `Process A` is used by `Process B`, `Process B` is used by `Process C`, and `Process C` is used by `Process A`, you cannot delete any of these processes individually. You must delete all of them simultaneously. Select all the processes, click **Actions**, and then select **Delete**.

1. On the **Explore** page, navigate to the object that you want to delete.
2. In the row that contains the project, folder, or asset, click **Actions** and select **Delete**.
3. To delete multiple assets, select the assets, click **Selected** and select **Delete**.

Tags

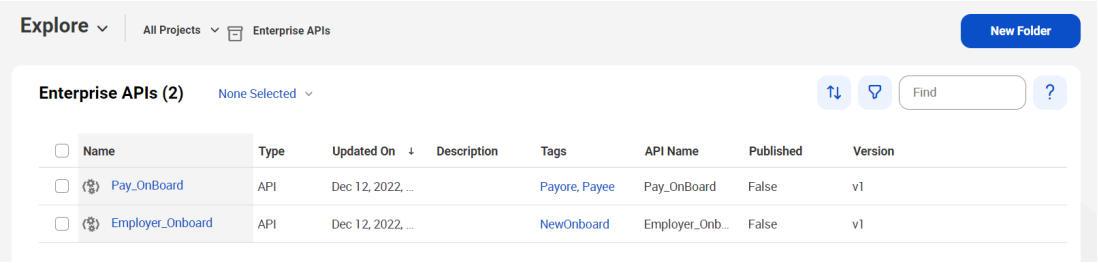
A tag is an asset property that you can use to group assets. Create tags to filter for assets that share a common attribute on the **Explore** page. You can assign tags to all asset types.

An asset can have multiple tags that add up to a maximum of 512 characters. A tag can have a maximum of 255 characters

You can find all of the assets that have a particular tag using one of the following methods:

- Click the name of the tag in the **Tags** column, in any row.
- Explore by tag, and then in the list of tags that shows on the page, click the name of the tag.

The following image shows the **Explore** page that lists tags created for the organization:



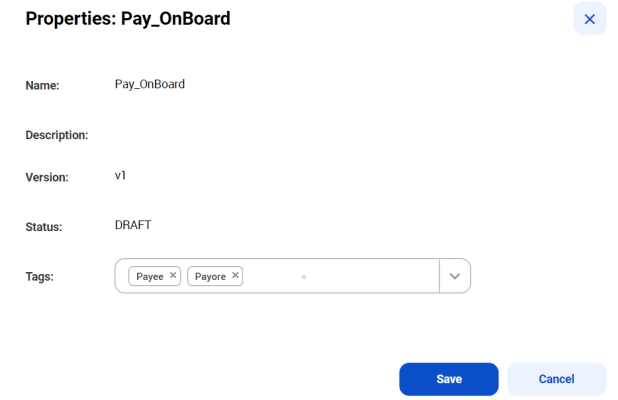
Creating tags

Use an asset's **Properties** dialog box to create and assign tags for that asset or to create tags to be available for future use.

Perform the following steps to create multiple tags without assigning them to an asset:

1. On the **Explore** page, browse by asset type.
2. In a row that contains an asset, click **Actions** and select **Properties**.
3. In the **Tags** field, enter the name of a tag that you want to create, and then press Enter.

   A tag can have a maximum of 255 characters.
4. Continue to enter the desired tags. Press Enter after each tag name to add it to the tag list.



5. After you have entered the tags, delete the tags from the **Tags** field so that the asset does not become associated with the tags. The tags will still appear in the list of available tags.
6. Click **Save**.

Assigning tags

You can assign a tag to one asset at a time or assign a tag to multiple assets at the same time. You can also assign multiple tags to one asset.

When you assign tags to an asset, you can choose an existing tag or create a new one.

1. On the **Explore** page, navigate to the asset or assets.
2. Perform one of the following tasks depending on whether you want to assign tags to one asset or assign tags to multiple assets at the same time.
   - To assign tags to one asset, in the row that contains the asset, click **Actions** and select **Properties**.
   - To assign tags to multiple assets at the same time, in the row for each asset, select the check box. After you have selected all of the assets, from the Selection menu, select **Tags**.
3. Select an existing tag or enter the name of a new tag.

   Continue adding tags or creating new tags until you have assigned all of the desired tags.
4. Click **Save**.

Editing and deleting tags

You can edit or delete a tag on the **Explore** page.

Edit a tag name or description in the tag properties. When you edit a tag, the properties for associated assets update as well. If you delete a tag, the tag no longer appears in the asset properties.

1. On the **Explore** page, browse by tags.
2. In the row that contains the tag, perform one of the following tasks:
   - To edit a tag, click **Actions** and select **Edit**. After you make your changes, click **Save**.
   - To delete a tag, click **Actions** and select **Delete**.

Assets

API Center assets include APIs and processes, such as Application Integration processes.

Create the assets you need to satisfy your business needs. A template is available for REST API assets. You can use the template as-is, or you can use it as a head start and customize it to meet your needs.

You can use assets in multiple projects. All assets must be part of at least one project. You can't delete assets that has dependencies on published APIs.

To prepare an existing API design for import or export, you must make the API design exportable. Open the API design, click on **Make Exportable**, then edit and save the existing API.

Asset dependencies

You can view the dependencies for an asset. You might want to view asset dependencies before performing certain operations on an asset.

For example, you can't delete an asset if another asset depends on the asset. You must first delete the dependent assets and then delete the asset.

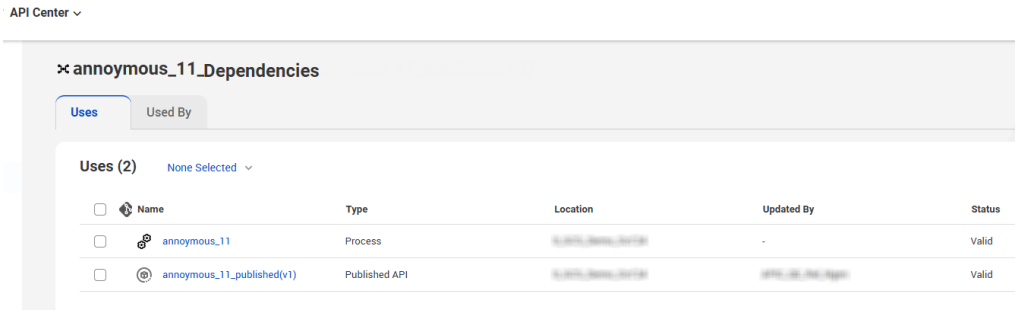
Perform the following steps to find the dependent assets of an asset:

1. On the **Explore** page, click **Actions** in the row that contains the asset.
2. Select **Show Dependencies**.
   The **Dependencies** page opens showing the **Uses** tab by default.

   The following table explains the available tabs on the **Dependencies** page:

Tabs	Description
Uses	Lists the assets that the selected asset uses. For example, a managed API is dependent on its parent assets, the published API, and API design.
Used By	Lists the assets that use the selected asset. For example, a policy might be used by a managed API.

   The following image shows the **Dependencies** page for a managed API with the **Uses** tab open:



   To drill down to the lowest level dependency, you can continue to view dependencies for each asset that appears on the **Dependencies** page. At the top of the **Dependencies** page, a breadcrumb shows the chain of dependencies.

The published APIs and managed APIs automatically inherit their location from the source asset.

For APIs created from a designed API design, API Center creates the published APIs and managed APIs in the same location as the original API design. Similarly, if you create a published API that does not originate from a design API, API Center uses the location of the source process to determine where to save the published and managed APIs.

CHAPTER 2

Asset migration

You can migrate API Center assets from one organization to another organization. To migrate assets, you export the assets from the source organization, and then import the assets into the target organization.

You can import and export the following types of assets:

- Design APIs
- Policies
- Published APIs
- Managed APIs
- API groups
- Managed API groups
- Application Integration processes and dependent objects

You can export single assets, groups of assets, or all the assets in a project. If you export a project or folder, the file structure remains intact so that when you perform the import in the target organization, you can duplicate the original structure.

To export or import assets in a sub-organization, log in to the sub-organization.

Asset migration requirements

To migrate assets from one organization to another, the organizations and your user accounts in the organizations must meet certain requirements.

Ensure that you meet the following requirements:

- You have a user account in the source and target organizations with a role that has import and export privileges, such as the Admin or Designer role.
- The source and target organizations have the required license to import and export assets.
- The target organization has the required licenses for the assets that you want to import.
- The target organization uses the same version or a newer version of IDMC. The versions might differ temporarily if the organizations aren't on the same POD (Point of Delivery) during an IDMC upgrade.

Asset migration user roles

Before exporting or importing API Center assets, ensure that you are assigned the Admin, Designer, Policy Manager, Deployer, or Monitor user role.

The following table lists the assets that you can export or import based on the user role assigned:

Role name	Access privileges	Assets that can be exported	Assets that can be imported
Admin	Has full access to both export and import tabs	- Policies - Design APIs - Application Integration processes - Dependent objects	- Policies - Design APIs - Application Integration processes - Dependent objects
Designer	Has full access to both export and import tabs	- Policies - Design APIs - Application Integration processes - Dependent objects	- Design APIs - Application Integration processes
Policy Manager	Has full access to both export and import tabs	Policies	Policies

Dependent assets

Dependent assets are assets that are required by other assets.

When you set up an export, you have the option to include or exclude dependent assets in the export file. The dependent assets must exist either in the export file or in the target organization, else the import fails.

You might want to include dependent assets if they do not exist in the target organization. Or, you might want to include dependent assets if you want to replace the dependent assets in the target organization with updated versions from the source organization. If you choose to include dependent assets, the export file includes dependent assets for all of the assets that you include in the export. When you configure the import, you can choose which dependent assets to import.

You might want to exclude an asset's dependent assets if the objects exist in the target organization and you do not want to replace them.

When you import a set of assets, the import fails if you unselect an asset that is a mandatory dependency of another asset. For example, if a managed API depends on a published API, but you unselect the published API during import, the import fails with an error indicating that the required dependent object is missing. To successfully import the assets, you must select all the dependent assets before you proceed with the import.

When you import assets for the first time, all dependencies must be selected for the import to succeed. However, when you import the same assets again in overwrite mode, the dependencies already exist in your target system. In this scenario, you can unselect any of the dependent assets, and the import will still succeed because the required dependencies are already present in your target system.

View dependent assets

When you make an asset exportable, API Center automatically makes its related assets exportable. You can view the complete list of all dependencies to see how they are connected.

The following table lists the dependent assets of an exportable asset:

Asset Type	Dependent Asset
Policy	• Design APIs • Published APIs (latest revision) • Managed APIs
Design API	• Policies • Published API (latest revision) • Managed APIs
Published API	• Policies • Design API • Managed APIs
Managed API	• Policies • Published API • Design API
API group	• Policies • Design API • Published APIs • Managed APIs
Managed API group	• Policies • Design API • Published APIs • Managed APIs • API groups

API Center creates all the assets in the File Repository Service (FRS) by default. While all the assets reside in FRS, they are either ready for export or not ready for export. An asset inherits its status from its source when you create it:

- If you create an asset from a source that is ready for export, the new asset is also ready for export.
- If you create an asset from a source that is not ready for export, the new asset is also not ready for export. When you view the dependencies for such an asset on the **Explore** page, the page appears empty.

For example, if you create a managed API from a published API that is not ready for export, the new managed API is also not ready for export. When you make the managed API ready for export, API Center automatically makes the underlying parent asset and all its dependencies ready for export as well.

Asset export

When you export assets, IDMC creates an export ZIP file that contains the assets that you selected for export.

You can select individual assets to export, or you can select an entire project or folder. When you export a project or folder, the export file includes all of the assets in the project or folder.

To export an asset, you need the following privileges and permissions:

- Your user role must have privileges to export assets.
- You must have read permission on the asset.

Note: Informatica recommends that you include no more than 800 objects in an export file.

Make assets exportable

You need to make the assets exportable first to ensure that the assets and their dependent objects are properly prepared for a smooth and error-free export or import process. The ability to export and import assets gives you the flexibility to transfer assets across organizations without needing to re-create them.

If you are assigned an Administrator or Deployer role, you can make the assets exportable or ready to export.

If you are assigned the Administrator role, you can export the following asset types from an organization and import them into another organization:

- Design APIs
- Policies
- Published APIs
- Managed APIs
- API groups
- Managed API groups
- Application Integration processes and dependent objects

When you log in to API Center, you see that certain assets marked with the  symbol. This icon indicates that you can't export the asset. This type of asset doesn't appear on the **Explore** page.

Note: If you had created APIs or policies after October 2025, the assets are automatically enabled for export. However, if you had created APIs or policies before October 2025, you will need to use the **Make Exportable** option to prepare and mark the assets ready for export or import.

Making design APIs exportable

You can refresh existing design APIs to make them export-ready without changing other properties. Starting from October 2025, editing and saving an existing design API automatically prepares it for export. After you create and save an API, any non-exportable policy linked to that API is automatically synced and becomes export-ready.

Perform the following steps to make an API exportable:

1. Open a design API.
2. Click **Make Exportable** on the icon toolbar.
   A message appears confirming that the API is made exportable.

3. Alternatively, if you edit the design API properties and save the API, the API is automatically made exportable.

Making policies exportable

You can refresh existing policies to update their status as exportable without changing other properties. You can also edit and save existing policies to auto-update them to be export-ready.

Perform the following steps to make a policy exportable:

1. On the **Policies** page, click the **Actions** menu on the row of the selected policy, and then click **Make Exportable**.
   A message appears confirming that the policy is made exportable.
2. Alternatively, if you edit the policy properties and save the policy, the policy is automatically made exportable.

You can use the **Make All Exportable** option to make all the policies exportable in one action. This option eliminates the need to mark each policy individually, saving you time when managing multiple policies in your organization.

Making published APIs exportable

On the **Published API** page, click the **Actions** menu on the row of the selected published API, and then click **Make Exportable**.

A message appears confirming that the published API is ready to export.

Making managed APIs exportable

On the **Managed API** page, click the **Actions** menu on the row of the selected managed API, and then click **Make Exportable**.

A message appears confirming that the managed API is ready to export.

Making API groups exportable

On the **API Groups** page, click the **Actions** menu on the row of the selected API group, and then click **Make Exportable**.

A message appears confirming that the API group is ready to export.

API group export location

An API group contains operations from multiple published APIs, and each operation can have different processes located in different folders. When you export an API group, API Center places it in the default location. You can then use the move and rename options to relocate it as needed.

Making managed API groups exportable

On the **Managed API Groups** page, click the **Actions** menu on the row of the selected managed API group, and then click **Make Exportable**.

A message appears confirming that the managed API group is ready to export.

Managed API group export location

When you export a managed API group, API Center checks the folder where the API group is located and uses the same folder location for exporting the managed API group.

Export files

The export file retains the file structure of the source organization's **Explore** page for projects, folders, and assets. The export file includes a spreadsheet that lists the objects within the file.

Object names

Each asset is contained in a ZIP file along with its associated metadata and JSON file. The zip file includes the asset name appended by the asset type.

Exporting assets

You can select a single asset, multiple assets, or a project to export.

You can select one or more assets within a folder. Or, select a project or folder to export all of its assets.

1. On the **Explore** page, navigate to the assets that you want to export.
2. Select the assets.

   To export a single asset or project, select the asset or project.

   To export multiple assets, select the check box to the left of each asset that you want to export. Or, select the check box for each project or folder that contains the assets that you want to export.
3. From the selection menu, select **Export**.
4. On the **Export Assets** page, change the job name or accept the default name.
5. Select whether to include dependent objects for the assets.

   Note: The **Export Assets** page lists the assets in the export operation and indicates the total number of assets to export. You can sort and filter the list.
6. Click **Export**.
7. To see the progress of the job, go to **My Import/Export Logs** page and click the **Export** tab.

   Click the name of the log to open the log details page.
8. To download the export file, click **Download** next to the export job on the **My Import/Export Logs** page.

Exporting duplicate assets

If you are assigned an Administrator or Deployer role, you can make an asset with the same name exportable in the same location.

You can export and rename the following asset types:

- Published APIs
- Managed APIs
- API groups
- Managed API groups

When you save an asset with the same name in the same folder, API Center appends a timestamp to the duplicate asset. You can rename the asset.

1. On the **Explore** page, navigate to the asset that you want to rename.
2. Click the **Actions** menu, and then select **Rename**.
3. Update the name in the **Rename Asset** dialog box and click **Save**.

Asset import

You can import all of the assets in an export file or select the assets that you want to import.

When you import assets, you specify the following information:

- The assets in the export file that you want to import and the projects in which to import them.
- Whether to overwrite assets in the target project with the assets in the export file when there is a name conflict.
- Whether to merge tags when you overwrite assets in the target project.

To import an asset, you need the following privileges and permissions:

- Your user role must have privileges to import assets.
- If you import an asset into the target project as a new asset, you must have create, update, and read permissions on the asset.
- If you overwrite an asset in the target project, you must have update and read permissions on the asset.

The **Import Assets** page lists the assets that are in the export file. You can select which assets you want to import, and then specify which project to import the assets to. You can accept the default project, which is the same project name as the source project, or you can select a different project. If the project does not exist in the target organization, IDMC creates it.

Importing assets

Import assets from an IDMC export file.

When you import a project, you can import the entire project or import the assets that you select.

Note: When you import a Data Quality design API, you must create a deployment with the same name as in the source system by logging into the Data Quality service.

1. Log in to the target organization.
2. On the **Explore** page, click **Import**.

   The **Import Assets** page appears.
3. On the **Select Import File** section, navigate to the export file and click **Open**, or drag and drop the zip file from your saved location.

   The **Import Assets** page lists the assets in the file.
4. Optionally, change the import job name.
5. Choose whether to overwrite existing assets with the assets in the import.
   - If you choose to overwrite existing assets, when an asset has the same name as an asset in the target project, the asset replaces the existing asset in the target project.
   - If you do not choose this option and an asset with the same name exists in the target project, the existing asset is retained.
6. Select the assets to import.

   If the export file contains a project and you want to import the entire project, select all of the assets. IDMC creates the project in the target organization.
7. Select the target project or accept the default.
8. Click **Test** to see the potential results of the import.

   In the **Select Assets** area, the status for each asset shows the action that the service performs when you import the files.

9. If necessary, revise your selections to resolve any issues in the test results.
10. Click **Import**.

    You can see the progress of the import on the **Import** tab of the **My Import/Export Logs** page. When the import process is complete, a message appears in **Notifications**. Click the link in the message to open the log details page and see the results of the import.

    Note: You can't import a policy into a different folder if a policy with the same name already exists in another folder in the target system. The policy import fails due to a naming conflict.

Asset name conflicts

You can specify how API Center handles asset name conflicts when the export file contains assets with the same name as assets in the target project. You can choose whether to overwrite the assets in the target project or use the existing assets in the target project.

To see how the import handles any asset name conflict before you start the import job, you can test the import on the **Import Assets** page. The import action displays in the **Status** column for each asset. You can filter the list of assets by asset name, asset type, or status.

Rules and guidelines for importing designed APIs

Consider the following rules and guidelines when import designed APIs:

- When you import a designed API, it is always imported in the draft state, regardless of its state in the source system. The imported designed API appears as draft in the target system.

Rules and guidelines for importing published APIs

Consider the following rules and guidelines when import published APIs:

- When you import a published API without its corresponding designed API, and the designed API is in the published state, the import fails.
  You can't have a published API without a designed API, and you can't create or update a published API that is linked to a designed API in the published state. If you have updated your published API, the designed API and published API can become out of sync. API Center prevents such import to avoid showing a designed API in the published state while its corresponding published API has been updated.
- Even if your designed API and published API are in sync, you must still publish the API for the designed API to return to the published state.

Rules and guidelines for importing managed APIs

Consider the following rules and guidelines when you import managed APIs:

- After you import a managed API, you might see the **Updates Available** icon next to the published API name on the **Managed API** tab in the **API Console** page.
  This symbol indicates that an update is available. Since API Center considers the import of a managed API to be an update, this symbol will always appear after you import a managed API.

  Additionally, the update available notification can also indicate that the source published API has a new version, while the corresponding managed API is on an older version.

  Click on the **Actions** menu of the managed API and click **View Updates**. You can then review the changes, if any, and click **Accept** to finalize the import and clear the notification.

- You can't import a managed API if its target asset is in an active, shared, or deprecated state. Any import attempt on an API in these states might cause accidental updates to a live API that customers might be using. To successfully import the managed API, you must first change its status to inactive or create. After importing the changes, you can test the API, and then set its status back to active.
- When you import a managed API, the import fails if the URL context of the managed API already exists in the target organization, even if it belongs to a different managed API. The URL context must be unique within an organization. To resolve this issue, you must change the URL context either in the source system before you export or in the target system before you import.

Downloading an import or export log

When an import or export instance is complete, you can download the log file. Download the log file from the import or export instance details.

1. Select **My Import/Export Logs** to open your import/export logs.
2. Click the **Import** or **Export** tab.
3. Click the instance name to open the import or export instance details.
4. Click **Download Import Log** or **Download Export Log**.

CHAPTER 3

Source control

You can use a Git source control repository to manage and track changes made to objects such as projects, folders, and assets.

The following table lists the source control repositories that you can use:

Repository	Self-hosted	Cloud-hosted
Atlassian Bitbucket	Supported	Supported for Data Integration assets only
GitHub	Supported	Supported
GitLab	Supported	Supported
Microsoft Azure DevOps	-	Supported

Note the following guidelines for using repositories:

- You can't use cloud-hosted Bitbucket or cloud-hosted GitLab if the organization is subject to FedRAMP.
- If you use a GitLab repository, various source control actions such as pull and compare actions might show incorrect commit history.

You can use source control to enable version management for the IDMC objects that appear on the **Explore** page, except for Data Integration bundles. You cannot apply source control to objects that don't appear on the **Explore** page such as runtime environments or connections. The source control repository structure mirrors the structure in the organization. By default, **Explore** is the top level directory.

Each time a user performs an action on an object that is source controlled, IDMC logs the action. You can view source control logs in the Monitor service. For more information, see *Monitoring source control logs* in the Monitor help.

To use source control, the following prerequisites must be met:

- The organization has the appropriate IDMC licenses to use source control.
- The organization administrator has configured a connection between the source control repository and the IDMC organization.
- Your user role has privileges to use the IDMC source control feature.
- You've entered your source control repository user credentials in IDMC.

For information about configuring a connection between a Git source control repository and IDMC, see the Administrator help.

Note: To optimize performance and avoid infrastructure limitations, include no more than 1,000 objects in a container such as a project or folder when you use a source control repository.

Source control actions

You can perform the following actions on source-controlled objects such as projects, folders, and assets:

Pull an object.

Pull an object to add it to the organization or update a project with the version in the source control repository.

Check out an object.

Check out an object that you want to work on. When you check out an object, the object locks so that other users cannot make changes to it.

Check in an object.

Check in an object to add it to the source control repository or update the source control repository with the latest version of the object. When you check in an object, the lock releases.

Delete an object.

Delete an object from the organization and the source control repository. Before you can delete an object, you must check it out.

Restore an object version.

Restore an object to a previous version.

Undo a checkout.

Undo a checkout if you don't want to save the changes you made to the object. When you undo a checkout, the object reverts to the last source control version.

Unlink an object.

Unlink an object if you no longer want the object in the organization to stay in sync with the object in the source control repository.

Note: Some organizations do not have permission to update the source control repository. If your organization cannot update the repository, you can perform a pull action to get a specified version of the IDMC objects. However, you cannot perform other source control actions such as checking out and checking in objects.

To check out, pull, unlink, or undo a checkout for an object, you must have update permissions on the object.

Source control and the Git repository

If you usually work directly in a Git source control repository, you might notice a few differences between using source control in IDMC and working directly in the repository.

Note the following differences:

- A pull action in IDMC is the same as a Git pull command. However, a pull in IDMC cannot merge changes.
- A checkout action in IDMC locks an object so that no one else can check it out or change it.
- A check-in action in IDMC is the same as a Git commit command and push command combined. Use a check-in to add IDMC objects to the source control repository and to commit changes to the repository.

Configuring repository access

To work with source-controlled objects, specify your repository credentials in IDMC.

Your credentials can include a personal access token or API token, depending on the repository service that you use. If your organization's repository is configured for OAuth access, you can enable OAuth access instead of providing a personal access token or app password.

Personal access tokens and API tokens must be configured to enable full control of private repositories. For information about generating tokens, see the GitHub, GitLab, Azure DevOps Git, or Bitbucket help.

Perform the following steps to configure access to the repository:

1. Click the **User** icon on the toolbar and then select **Settings**.

   Note: Depending on which service you are in, the user settings page might appear different and show different names for the sections.
2. In the **Source Control Access** section, perform one of the following tasks:
   - Enter your repository credentials.
   - Enable OAuth access to the repository.
     If you haven't already authorized access, a Git access app appears for GitHub and Azure DevOps Git repositories. Select to authorize access for IDMC.
3. Click **Save**.

Project-level repositories

If you're assigned a role with the Configure Custom Repository Source Control privilege, you can specify a source control repository to use at the project level, instead of using the global source control repository that's set up for your organization.

By default, when you perform a source control action on a project or folder, the action uses the global source control repository that's set up by the organization administrator. If the organization administrator enables project-level repositories, you can link a project to a repository other than the global repository.

If you use project-level repositories, be sure to add the Repository URL and Repository Branch columns to the Explore page.

Note: Before you link a project to a custom repository, make sure that other users who need access to the project have access to the repository.

Linking a project to a source control repository

You can link a project to a source control repository other than the global repository.

1. On the **Explore** page, in the row that contains the project, click Actions and select **Source Control Properties**.
2. Enter the Git repository URL and branch name that you want to use, and then click **Save**.
3. Click **Save** again to confirm that you want to use a repository other than the global repository.

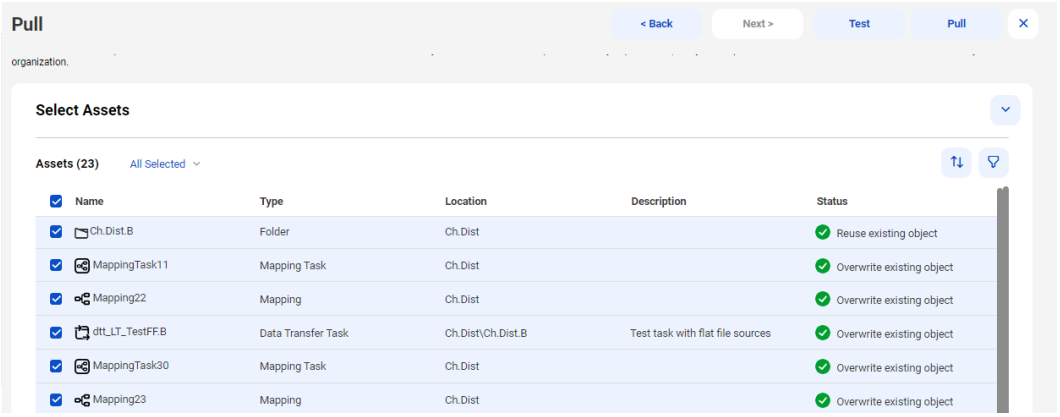
   After you click **Save**, only users who have access to the repository can check out and commit the project.

Source control pulls

Pull an object to add it to the organization or replace the version of the object in the organization with the version in the source control repository. You can pull a project, folder, or an individual object that isn't checked out by another user.

Before a pull takes place, you can review the potential results of the pull. You can cancel the action or select objects to exclude from the pull.

The following image shows a preview page with potential results in the **Status** column:



You can also test the pull to see if any errors might occur. If any errors appear, you can exclude the objects from the pull or close the page to cancel the pull.

A pull doesn't include dependent objects. For example, in Data Integration, if you pull a mapping that uses a saved query, the pull doesn't include the saved query.

A pull doesn't change the state of pulled objects in the organization. For example, if an object was checked out before the pull, it remains checked out after the pull.

Consider the following rules and guidelines:

- If you want to pull an object that uses a connection or runtime environment, be sure that the organization includes a connection and runtime environment before you perform the pull. When you select an object that uses a connection or runtime environment, you can select a runtime environment and connection to use on the test page.
- IDMC doesn't consider capitalization in object names. As a result, you can't pull a project that contains multiple assets that have the same name with different capitalization. For example, if you try to pull a project that contains an asset named "sales" and an asset named "SALES", you receive an error because a project or folder can't contain multiple assets with the same name.
- You can pull objects regardless of whether your organization can or cannot update the source control repository.

Project and folder pulls

You can pull all of the objects in a project or folder, or pull only the objects that changed in a particular commit hash.

You specify the version of the objects that you want to pull. The versions of an object that you can select are based on the object's current location in IDMC. For example, you check out version 5 of the m_customers object and move it to a project called NewCustomers. When you pull the object, the available versions do not include the versions of the object from the previous location.

If you pull a project or folder, the pull doesn't affect any objects in the project or folder that aren't source-controlled.

A pull updates the project or folder in the organization to be identical to the selected version of the project or folder in the repository. For example, if you pull an older version of a project and the project in the organization contains objects that were added in later versions, the pull deletes the objects.

Pulling an object

Use the pull action to update the organization with objects in the source control repository. You can pull a single asset, a project, or a folder. When you pull a project or folder, you can exclude any assets in the project or folder that you don't want to include in the pull.

1. To pull a project from the global repository, click **Pull from Global Git**. Select the project that contains the objects you want to pull and click **Next**.
2. To pull a project from a repository or branch that's configured at the project level, on the **Explore** page, select the asset or folder. Click **Actions** and select **Pull**.

   Note: You can't pull more than one project at a time.
3. To pull a single asset or folder, on the **Explore** page, select the asset or folder. Click **Actions** and select **Pull**.
4. For a project or folder, select the pull method that you want to use, and then click **Next**. You can use one of the following methods:
   - Pull all assets in a commit hash.
   - Only pull modified assets in a commit hash.
5. Select the Git hash that you want to pull and click **Next**.
6. Review the list of assets. You can clear the check box for any of the assets that you don't want to include in the pull. By default, all of the assets are selected.
7. Review the actions in the **Status** column for each asset. These actions will occur when the pull action is performed.
8. Optionally, click **Test** to see if errors might occur as a result of the pull action.
9. Click **Pull**.

   The pull action generates a log showing details of this action. You can view the log on the **Source Control Logs** page in Monitor.

Commit tags

When you check in assets to your source control repository, you can include a commit tag. The tag appears as an annotated tag in your Git repository.

You might want to include a commit tag to mark assets within a check-in that are ready for deployment. When you add a commit tag, the tag is applied to each asset in the commit. You can add one tag for each commit, and the tag must be unique.

When you pull assets, IDMC displays commit tags in the **Tag** column on the **Pull** page.

Note: The pull operation can take longer based on the number of commit tags you have in your repository.

Checking out and checking in objects

Check out an object so you can make changes to it. Check in the object when you want to update the source control repository with your changes. You can also check in objects to add them to the source control repository.

When you check out an object, the object locks so that other users can't make changes to it. When you check in an object, a new version of the object is created in the source control repository.

If you check in an object that's in a folder or project that isn't source controlled, the folder or project becomes source controlled. An asset can't reside in a source control repository unless it's in a container such as a project or folder.

If you import assets into an organization that uses source control, the assets must be checked out so that they can be overwritten.

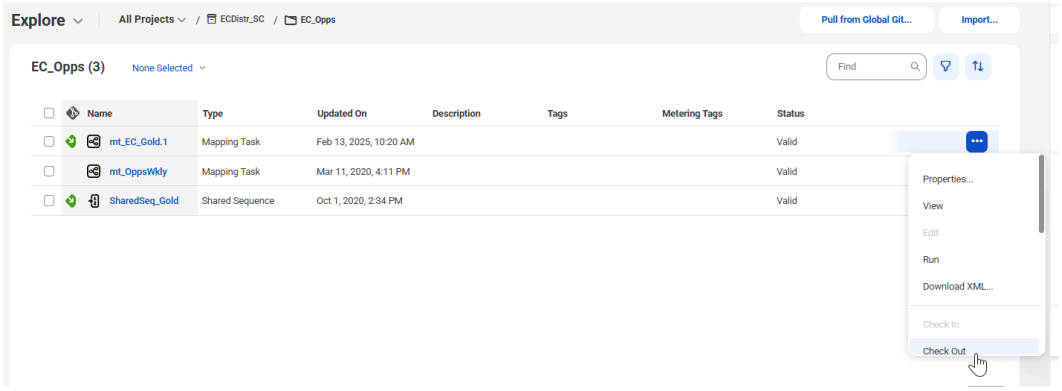
Note: The size of a check-in cannot exceed 50 MB.

Checking out an object

When you check out an object, the object is locked so that other users can't update it while you are making your changes.

You can check out individual objects, multiple objects, or a project or folder to check out all of the objects within the project or folder.

The following image shows the Check Out action is selected for the mt_EC_Gold.1 mapping task:



Before you check out an object, you might want to perform a pull to be sure you update the latest version of the object. For more information about the pull action, see "Source control pulls" on page 23

1. On the **Explore** page, navigate to the object you want to check out.
2. In the row that contains the object, click **Actions** and select **Check Out**.
3. If the checkout includes multiple objects, on the preview page, review the results in the **Status** column. If you want to exclude an object, clear the check box next to the object name.
4. Click **Check Out**.

Checking in an object

Check in an object to add the object to the source control repository or to update the source control repository with the latest version of the object in the organization.

If no changes were made to the object, the check-in isn't reflected in the source control history and a new version of the object isn't created in the source control repository.

Note: The size of a check-in can't exceed 50 MB.

1. On the **Explore** page, navigate to the object you want to check in.

   Note: If you use a cloud-hosted GitLab repository, a check-in action that includes more than 100 objects fails if GitLab doesn't process the check-in within around 30 minutes. We recommend that you check in fewer objects at once.
2. In the row that contains the object, click **Actions** and select **Check In**.
3. If the check-in includes multiple objects, on the preview page, review the results in the **Status** column. If you want to exclude an object, clear the check box next to the object name. If you want to cancel the action, close the page.
4. Add a summary and optionally, a description and commit tag.

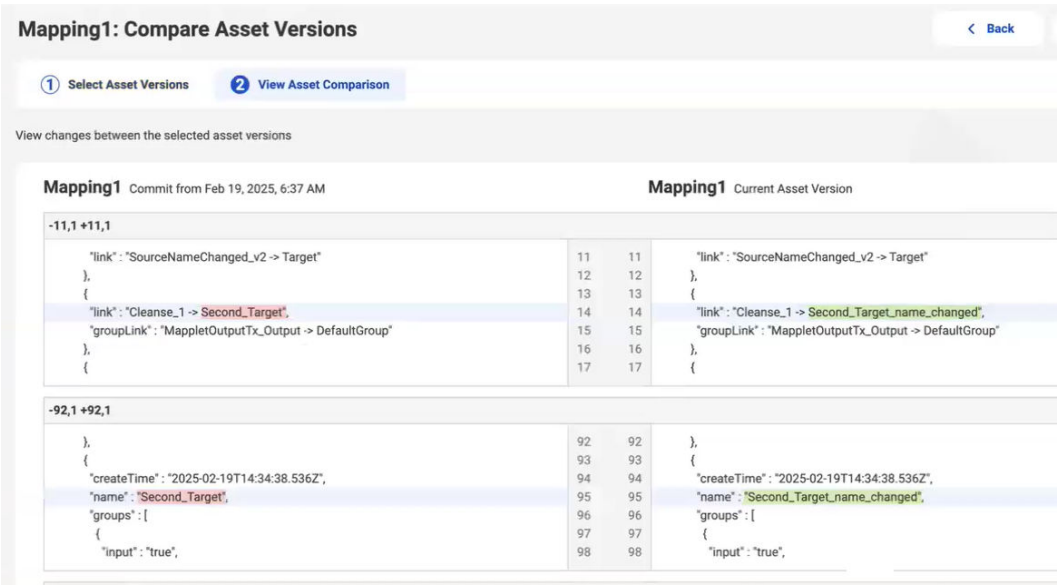
   A summary is required and has a max length of 255 characters.
5. Click **OK**.

Comparing asset versions

You can compare two versions of a source-controlled asset to find out what's changed between the versions. You might want to do a comparison to troubleshoot an issue or share code changes for peer code reviews.

You can compare the current version of an asset that hasn't been committed to the repository yet to a version in the repository, or compare two versions that were committed to the repository.

The **Compare Asset Versions** page displays the selected versions in text format with highlights that indicate the differences as shown in the following image:



Git doesn't retain the commit hash history for assets that you rename or move. Consequently, you can't compare a renamed or moved asset to a version of the asset that uses its old name or location.

Note: If the result of the asset comparison includes 10,000 lines or more, an error message appears. To compare large assets, use the Git compare feature within the Git repository.

1. On the **Explore** page, navigate to the asset that you want to compare.
2. In the row that contains the asset, click **Actions** and then click **Compare Asset Version**.
3. To compare the current version to a version that's been committed to the repository, select **Compare the current asset version and a previous asset version**, and then select the previous asset version that you want to use.
4. To compare two previous versions, select **Compare two previous asset versions**, and then select the two versions that you want to compare.
5. Click **Next**.

Deleting an object

To delete a source controlled object, you delete it from the organization and from the source control repository.

You must check out an object before you can delete it.

You can't delete an object that's checked out by another user or delete a project or folder recursively.

1. On the **Explore** page, navigate to the object that you want to delete.
2. On the row that contains the object, click **Actions** and then click **Delete**.
3. To confirm that you want to delete the asset from the organization, click **Delete**.
4. Add a summary that describes the reason for the delete action and optionally, a description.

   A summary is required and has a max length of 255 characters.
5. To delete the object from the repository and complete the delete action, click **OK**.

Reverting to an older version

You might want to revert to a previous version of an object if you want to discard changes that were made to the object.

To revert to a previous version, perform a pull action and select the version that you want to restore in the organization.

If you revert the version of a project or folder and the project or folder in IDMC contains objects that are not in the repository's project or folder, the pull action deletes the additional objects if they are source controlled. If the objects are not source controlled, the action doesn't delete the additional objects.

For more information about the pull action, see “Source control pulls” on page 23.

Undoing a checkout

When you undo a checkout, the object reverts to the last version that was pulled. The undo checkout releases the lock so that the object is available for checkout.

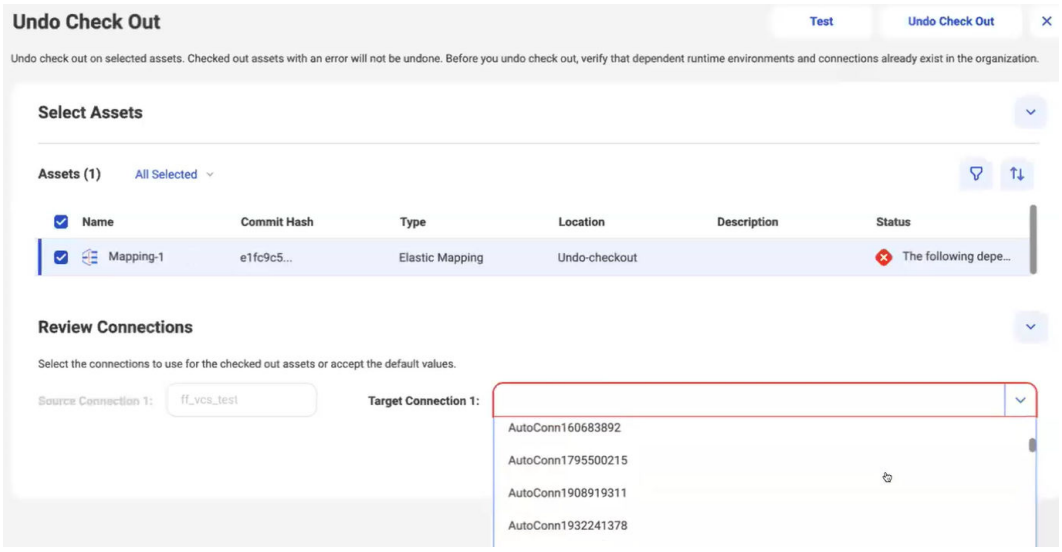
You can undo the checkout of individual objects, multiple objects, or a project or folder in a single checkout action. If you undo the checkout of a project or folder, you can select which objects within the project or folder to include or exclude. By default, all of the objects are included.

You can undo the checkout of any object that you have checked out. You can also undo the checkout of an object that has been checked out by another user if you have the Admin role or your user role has the Force Undo Checkout feature privilege for Administrator. You might need to undo the checkout of an object that has been checked out by another user if the user has checked out objects and goes on vacation or leaves the organization.

The object's version history won't include a record of the checkout and undo checkout actions. If you think you might need the changed version later, make a copy of the object before you undo the checkout.

When you undo the checkout of an asset that uses connections and runtime environments, you can use the current connection and runtime environment values or select a different connection or runtime environment to use.

The following image shows how you can select a different connection for the asset during the undo checkout action:



Note: If an object was moved or renamed after it was checked out, undoing the checkout restores the object's name and location to its name and location before it was checked out.

1. On the **Explore** page, navigate to the object.
2. In the row that contains the object, click **Actions** and select **Undo Check Out**.
3. If the undo checkout includes a project or folder, on the preview page, select the objects within the project or folder to exclude from the undo checkout action.
4. If applicable, review the selected connection and accept the default or select a different connection to use.
5. If applicable, review the selected runtime environment and accept the default or select a different runtime environment to use.
6. Click **Undo Check Out**.

Unlinking an object

You can unlink an object so that it's no longer source controlled.

Unlinking an object doesn't delete the object from the source control repository or the organization, but you can no longer update the repository for any changes you make to the object in the organization. If you decide to link the object in the future, you can check in the object to reestablish the link. If the name of the object or the path to the object has not changed, the checked in object becomes a new version of the object in the source control repository.

The object must be checked in before you can unlink it. You can unlink an object that's checked out by another user if you have the Admin role or your user role has the Force Undo Checkout feature privilege for the Administrator service.

You can't unlink a project or folder that contains source controlled objects. To unlink a project or folder, unlink each object within the project or folder first.

1. On the **Explore** page, navigate to the object that you want to unlink.
2. On the row that contains the object, click **Actions** and select **Unlink**.

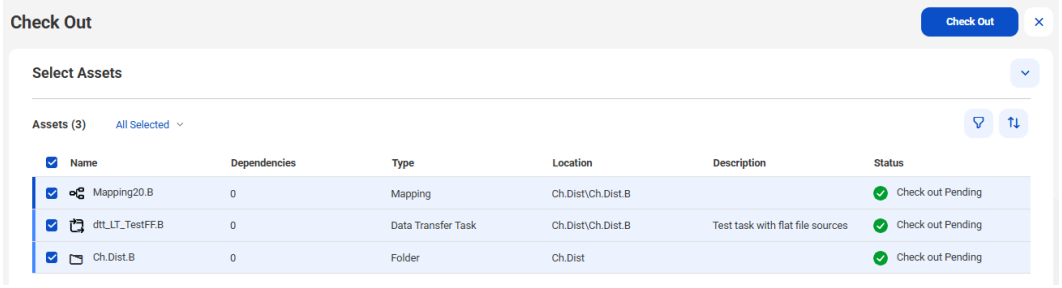
Working with multiple objects

You can select multiple objects to check in, check out, undo a checkout, pull, or unlink.

The total size of the selected objects cannot exceed 50 MB.

You can select a project or folder or select multiple objects within a project or folder. When you include multiple objects in a source control action, a preview page appears that shows you the expected results of the action if you proceed. If an object listed on the preview page is not source controlled, it will be ignored. If an object is checked out by another user or you do not have permission to update the object, the status on the preview page shows that the action will fail. You can opt to remove any of the objects before you continue.

For example, in the Default project, you select the Ch.Dist.B folder to check out. The preview page includes the Ch.Dist.B folder and the objects in the Ch.Dist.B folder. The following image shows the preview page:



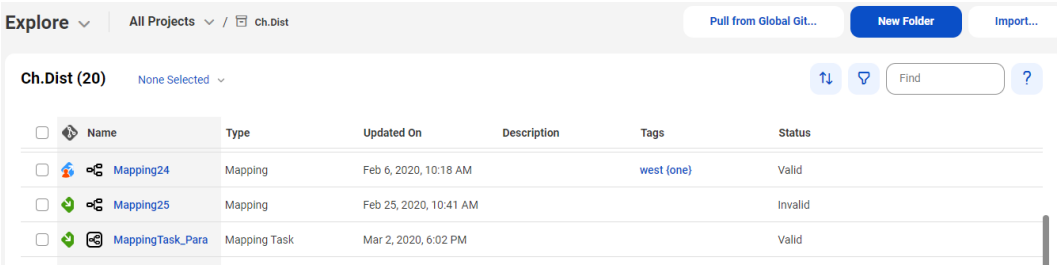
If you select multiple objects for a pull, the preview page lists all of the versions in the repository.

Note: You can't delete multiple objects in one transaction.

Viewing source control columns on the Explore page

If your organization has any objects that are source controlled, the **Explore** page displays an additional column that indicates whether an object is checked in, checked out, or is not source controlled.

In the following image, the blue and red icon indicates that the Mapping24 object is checked out:

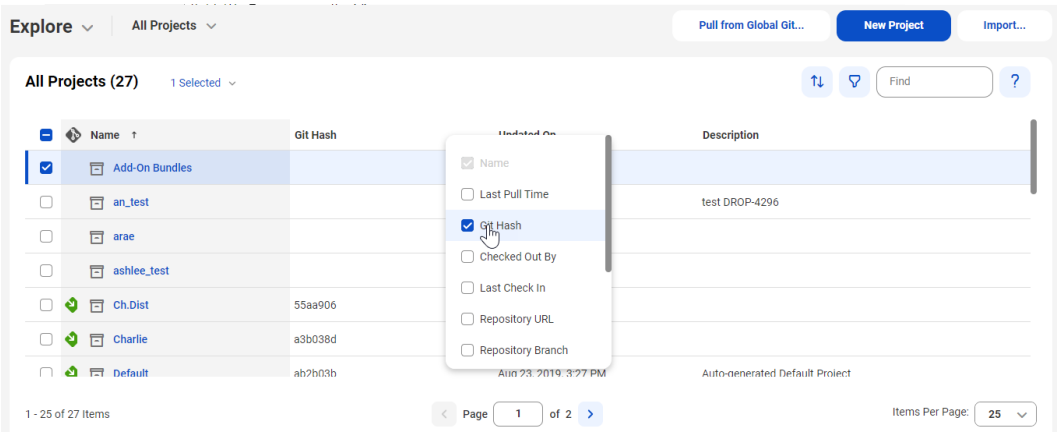


Additional source control columns are available but are not displayed by default. You might want to customize the **Explore** page to show all of the source control-related columns. This way you can easily identify source controlled objects, and you can use the columns to filter and sort source controlled objects on the page.

You can add the following source control-related columns to the **Explore** page:

- Last Pull Time
- Checked Out By
- Last Check In
- Git Hash
- Repository URL
- Repository Branch

To display these columns, right-click a column heading and select the columns that you want to add, as shown in the following image:



Source control best practices

To use source control effectively, use the following guidelines as best practices.

Adhere to the following guidelines as you develop and work with assets:

Guidelines for managing dependencies

Use the following guidelines to manage assets with dependencies:

- Create connections and runtime environments before you pull assets from the repository.

  When required connections and runtime environments exist in the target organization, you can run tasks immediately after you pull them from the repository.

- Ensure that reusable assets such as mappings and components are present in the repository before you use them.

  IDMC does not allow you to save an asset such as a mapping task when the dependent mapping does not exist in the organization.

Guidelines for checking in and checking out assets

Use the following guidelines when you check in and check out assets:

- When you rename or move an asset, check out the asset's first-level dependent assets and include them in the same check-in.

  For example, if you want to rename a mapping that a mapping task uses, and the mapping task is used in a taskflow, check out the mapping and the mapping task. You don't need to check out the taskflow. After you rename the mapping, check in the mapping and the mapping task in one check-in action.

- Enter comments when you check in assets.

  When you check in assets, you might enter a release tag name in the **Summary** field and enter more descriptive comments in the **Description** field. When you do this, the **Git Summary** field in IDMC shows the release tag that is associated with the asset.

- When you check in multiple assets at one time, limit the number of assets to 800 or fewer.

  Checking in more than 800 assets at one time can degrade performance between IDMC and the Git repository service.

INDEX

A

assets
- assigning tags 9
- comparing versions 26
- creating tags 8
- deleting 7
- exporting 14
- importing 17
- migration requirements 11
- renaming 7
- source control 20
- tags 8

Azure DevOps user credentials 22

B

Bitbucket user credentials 22

C

comparing asset versions 26
connections
- importing 17

creating
- tags 8

D

deleting
- assets 7
- folders 7
- projects 7
- tags 9

E

editing
- tags 9

Explore page
- source control columns 30
- tags 8

exporting
- assets 14
- export file 16

exporting assets
- requirements 11

F

folders
- deleting 7

G

GitHub user credentials 22

I

importing
- assets 17
- connections 17
- name conflicts 17
- projects 17

importing assets
- requirements 11

P

projects
- deleting 7
- importing 17

R

renaming
- assets 7
- folders 7
- projects 7

S

source control
- actions 21
- best practices 31
- checking in and checking out objects 25
- checking in objects 26
- checking out objects 25
- comparing assets 26
- configuring access to the repository 22
- deleting objects 27
- Explore page columns 30
- Git commands 21
- project-level repositories 22
- pulling objects 23
- pulling objects from the repository 24
- pulling projects and folders 23
- reverting to previous versions 27
- selecting multiple objects 29

source control *(continued)*
supported objects 20
supported source control systems 20
undoing a checkout 28
unlinking objects 29
updating organization with repository versions 23

T

tags
assigning 9
creating 8
deleting 9
editing 9
properties 9
tasks
deleting 7